

Memory Management for LLM-based Agents: An OS-Inspired Perspective on Agent Memory

Yuan Xie

Student ID: 224040374

A Report Submitted for
CSC6032 Advanced Operating System

School of Data Science
The Chinese University of Hong Kong, Shenzhen
April 2026

Abstract

Large language model (LLM) agents are increasingly required to interact with users and environments over long horizons, but their built-in context windows remain inherently finite. This mismatch has made *memory management* a central systems problem for agent design. Rather than treating memory as a simple retrieval database, recent work increasingly views it as an operating-system-style subsystem that must support persistent storage, structured organization, dynamic update, efficient retrieval, and resource-aware reuse. This report studies this emerging perspective by integrating one recent survey on agent memory and three representative works: *A-Mem*, *MemoryOS*, and *Agentic Plan Caching* (APC). This report organizes these works around a common lifecycle of formation, evolution, and retrieval. A-Mem emphasizes self-organizing memory notes and evolving semantic links; MemoryOS designs a hierarchical memory operating system with short-term, mid-term, and long-term stores, plus paging- and heat-inspired management policies; APC focuses on serving-time efficiency by caching reusable planning templates and adapting them across semantically similar tasks. Through a comprehensive comparison, this report argues that these systems collectively indicate a broader direction: future agent memory should be treated as a first-class managed subsystem with explicit structures, policies, and efficiency objectives, much like memory hierarchies, paging, and caching in operating systems.

Contents

Abstract	i
1 Introduction	1
2 Background and Related Work	3
2.1 Why memory has become a systems problem for LLM agents	3
2.2 A survey view: forms, functions, and dynamics	4
2.3 Agent memory versus RAG and context engineering	5
2.4 OS-inspired lines of related work	6
3 A-Mem: Agentic Memory for LLM Agents	7
3.1 Motivation	7
3.2 Overall architecture	8
3.3 Method details	8
3.3.1 Structured note construction	8
3.3.2 Link generation through retrieve-then-reason	9
3.3.3 Memory evolution	9
3.3.4 Memory retrieval	10
3.3.5 Algorithmic summary	10
3.4 Experimental settings	11
3.5 Experimental results and discussion	11
4 MemoryOS: A Memory Operating System for AI Agents	14
4.1 Motivation	14
4.2 Architecture overview	14
4.3 Method details	15
4.3.1 Short-Term Memory and dialogue-chain pages	15
4.3.2 Mid-Term Memory with segmented paging	15
4.3.3 Long-Term Personal Memory	16
4.3.4 Memory updating and heat-based management	16
4.3.5 Retrieval and response generation	17

4.3.6	Algorithmic summary	17
4.4	Experimental settings	18
4.5	Experimental results and discussion	18
5	Agentic Plan Caching: Test-Time Memory for Serving Efficiency	20
5.1	Motivation	20
5.2	Method overview	20
5.3	Method details	22
5.3.1	Keyword-based cache lookup	22
5.3.2	Plan template extraction	22
5.3.3	Template adaptation on cache hits	23
5.3.4	Algorithmic summary	23
5.4	Experimental settings	23
5.5	Experimental results and discussion	24
6	Conclusion and Future Work	26
	Bibliography	28

List of Figures

2.1	Survey overview of agent memory organized by forms, functions, and dynamics.	4
3.1	Comparison between a traditional memory system and A-Mem.	7
3.2	A-Mem architecture overview including note construction, link generation, memory evolution, and retrieval.	8
3.3	Impact of retrieval parameter k across task categories.	12
4.1	MemoryOS architecture with STM, MTM, and LPM, plus storage, update, retrieval, and generation modules.	15
4.2	Ablation study on GVD and LoCoMo benchmark datasets.	19
5.1	Motivation for APC: limitations of standard caching for Plan-Act agents.	21
5.2	Agentic plan caching framework.	21
5.3	Query-based versus keyword-based cache search.	22
5.4	Main APC results across baselines.	24
5.5	Accuracy comparison across caching methods.	25

List of Tables

3.1	Representative A-Mem results on LoCoMo.	12
3.2	Scaling analysis for different memory systems.	13
4.1	LoCoMo comparison for MemoryOS.	18
5.1	Cost breakdown for APC on FinanceBench and TabMWP	24
5.2	Fuzzy keyword matching results in APC under different similarity thresholds.	25

Chapter 1

Introduction

Large language models have recently evolved from passive next-token predictors into agentic systems that can reason, plan, invoke tools, interact with software environments, and maintain sustained dialogues across extended time horizons. This transition has fundamentally changed the role of memory. In standard single-shot prompting, historical context is transient and is simply appended to the input sequence. By contrast, in LLM-based agents, memory becomes a persistent computational resource that underpins continuity, personalization, task accumulation, and long-horizon decision making. As a result, memory is no longer merely an auxiliary retrieval component; it has become a central systems problem.

A key challenge is that the internal context window of an LLM is finite. Even when modern models support long contexts, they still face nontrivial issues: irrelevant history can degrade performance, token budgets increase monetary and latency costs, and the system cannot naively retain every prior interaction indefinitely. Consequently, an agent must decide *what* to store, *how* to organize it, *when* to update or forget it, and *how* to retrieve the most useful subset under resource constraints. These questions are highly reminiscent of classic operating systems problems such as hierarchical storage, paging, compaction, cache replacement, and access scheduling.

This report studies memory management for LLM-based agents through an operating-systems lens. The central argument is that recent agent memory systems increasingly resemble OS subsystems: they define explicit storage units, movement policies across memory tiers, update rules for memory consolidation, and retrieval or caching strategies that balance utility against computational cost. To support this perspective, the report synthesizes four materials provided for the course presentation: a recent survey on agent memory and three representative papers, namely A-Mem: Agentic Memory for LLM Agents [11], Memory OS of AI Agent [4], and Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents [13].

The survey paper provides a broad conceptual framework by organizing agent memory along three dimensions: forms, functions, and dynamics [2]. This formulation is espe-

cially useful because it moves beyond vague short-term versus long-term distinctions and instead asks three concrete questions: what carries memory, why memory is needed, and how memory is formed, updated, and retrieved over time. The three focal systems in this report instantiate these ideas from different OS-inspired angles. A-Mem emphasizes dynamic organization and semantic interlinking of memory notes. MemoryOS explicitly adopts a hierarchical storage architecture with update and eviction policies inspired by operating systems. APC addresses a different but complementary issue: efficient reuse of historical *plans* during the planning stage of agents, analogous to test-time caching in serving systems.

The report is organized as follows. Chapter 2 introduces the background and related work, centering on the survey’s taxonomy and on the distinction between agent memory, retrieval-augmented generation, and context engineering. Chapters 3, 4, and 5 then analyze A-Mem, MemoryOS, and APC respectively, with detailed attention to their motivations, methods, equations, experimental settings, and empirical findings. Finally, Chapter 6 concludes by comparing the three systems and discussing how they jointly point toward a more principled OS-style theory of memory for intelligent agents.

Chapter 2

Background and Related Work

2.1 Why memory has become a systems problem for LLM agents

The need for memory in LLM agents arises from the gap between the static inference interface of language models and the dynamic, stateful nature of agentic interaction. A conventional LLM call is typically stateless: given an input sequence, it generates an output and discards the interaction unless the caller explicitly preserves and re-injects the relevant history. However, an agent deployed in realistic scenarios must often preserve user preferences, remember prior task states, track partial plans, maintain long-term knowledge about environments, and exploit historical experiences to improve future efficiency. These requirements naturally create a memory management problem.

From a systems perspective, this problem contains at least four coupled challenges. First, *storage representation*: memory may appear as raw text, summaries, embeddings, structured key-value records, note graphs, plan templates, or even latent or parametric updates. Second, *organization*: memories may be grouped by time, topic, task, semantic similarity, or user profile. Third, *update policy*: systems must decide when new interactions should create memory entries, when historical memories should be revised or merged, and when stale information should be removed. Fourth, *retrieval and reuse*: the system must determine how to recover the most useful historical information under token, latency, and cost constraints.

These design dimensions strongly resemble the concerns of operating systems. Just as an OS distinguishes between registers, caches, DRAM, and disk, an agent memory system may distinguish short-term working memory from medium-term topical memory and persistent long-term personal memory. Just as an OS uses paging or segmentation to move data across tiers, agent memory systems increasingly use promotion, eviction, summarization, and compression strategies. Finally, just as caches aim to accelerate access without changing correctness, some agent memory systems aim not primarily to

improve capability but to reduce the cost of repeated reasoning by reusing historical artifacts.

2.2 A survey view: forms, functions, and dynamics

The survey argues that the literature on agent memory has become both rapidly expanding and conceptually fragmented [2]. Traditional labels such as short-term and long-term memory are no longer sufficient to describe contemporary systems, because recent methods vary substantially in their representational substrate, intended role, and operational lifecycle. To address this, the survey proposes a unified framework based on three orthogonal dimensions: *forms*, *functions*, and *dynamics*.

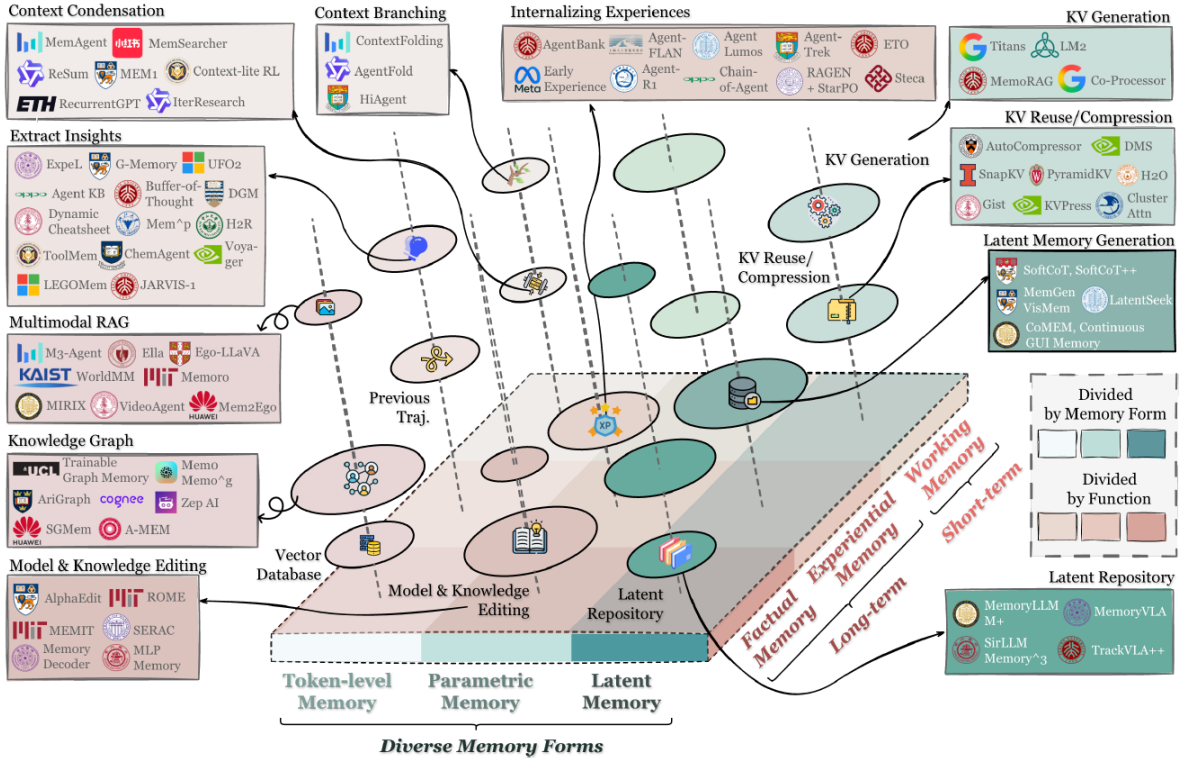


Figure 2.1: Survey overview of agent memory organized by forms, functions, and dynamics.

Forms. We can identify three dominant forms of memory realization: token-level memory, parametric memory, and latent memory [2]. Token-level memory stores explicit artifacts such as dialogue turns, summaries, knowledge notes, or retrieved passages. Parametric memory changes model parameters or uses model-internal trainable structures to encode long-term information. Latent memory uses hidden-state-like compressed carriers that may be generated, reused, or transformed across interactions. In this taxonomy, A-Mem and MemoryOS are primarily token-level external memory systems, whereas APC

can be interpreted as a structured form of test-time procedural memory implemented as reusable plan templates.

Functions. The role of memory can be refined into factual, experiential, and working memory [2]. Factual memory stores stable knowledge about users or environments; experiential memory stores episodes, strategies, or reusable skills acquired through past interactions; working memory supports immediate reasoning and multi-turn continuity. This distinction is particularly useful for the three focal systems in this report. MemoryOS explicitly targets factual and persona continuity across long conversations. A-Mem is more strongly aligned with experiential memory, because it stores and reorganizes historical experiences as interconnected notes. APC represents an efficiency-oriented form of experiential procedural memory, where historical planning traces are abstracted into reusable plan templates.

Dynamics. Finally, agent memory should be understood through explicit dynamic operations: *formation*, *evolution*, and *retrieval* [2]. Memory formation refers to transforming interaction artifacts into persistent candidates. Memory evolution describes the way stored memories are merged, revised, consolidated, or forgotten over time. Memory retrieval refers to generating task-aware queries and returning contextually relevant historical information during action selection. This dynamic perspective is especially important for distinguishing agent memory from static retrieval databases.

2.3 Agent memory versus RAG and context engineering

A major conceptual clarification in the survey is the difference between agent memory and related ideas such as retrieval-augmented generation (RAG) and context engineering [2]. Although these paradigms overlap, they are not identical.

RAG typically augments generation by retrieving relevant documents from an external store. In many traditional RAG systems, the knowledge base is comparatively static, and the main challenge lies in improving indexing and retrieval quality. Agent memory, by contrast, concerns persistent state accumulated across interactions, often involving dynamic write operations, revision of previous entries, and long-term personalization. In other words, many RAG systems focus on retrieval from a largely fixed corpus, whereas agent memory systems focus on lifecycle management of evolving memories.

Context engineering, meanwhile, refers to the careful construction of prompts or contextual inputs so that an LLM receives task-relevant information in a suitable format. Agent memory can support context engineering, but it is more than prompt assembly:

it requires explicit mechanisms for writing, updating, organizing, and selecting what should enter future prompts. This distinction becomes particularly clear in MemoryOS, where the authors propose a complete memory operating system with storage, updating, retrieval, and generation modules rather than treating memory as a single retrieval component [4].

2.4 OS-inspired lines of related work

The OS perspective on LLM memory did not emerge in isolation. A prominent earlier work is MemGPT, which explicitly frames LLMs as operating systems and uses virtual-memory-style paging to manage what remains in the context window versus what is swapped to external storage [10]. This line of thought shows that memory management for LLMs can be studied using principles long familiar in systems research: locality, paging, working sets, and tiered storage.

Other related work includes MemoryBank [14], which augments large language models with long-term memory and a forgetting mechanism, and Think-in-Memory (TiM) [6], which emphasizes recall and post-thinking in long-term memory settings. ReadAgent [5] introduces gist-based memory for very long contexts and is particularly relevant as a baseline in A-Mem. These works collectively indicate that long-context capability alone does not solve the memory problem; instead, effective systems require explicit structures and policies.

Within this broader landscape, the three focal papers of this report occupy complementary positions. A-Mem focuses on organization and evolution of memory notes. MemoryOS focuses on hierarchical management policies over multiple tiers. APC focuses on resource-efficient reuse during serving. Together, they provide a coherent snapshot of how memory in agent systems is increasingly being treated as an operating-systems-style subsystem.

Chapter 3

A-Mem: Agentic Memory for LLM Agents

3.1 Motivation

A-Mem begins from a critique of existing memory systems for LLM agents. According to the paper, many prior approaches enable basic storage and retrieval, but they still require developers to predefine memory structures, storage points, and retrieval timing within the workflow [11]. Even when graph databases are introduced, the authors argue that those systems remain constrained by static schemas and predefined relationships. Such rigidity becomes problematic in open-ended tasks where new experiences may suggest novel semantic connections or may require reinterpretation of old memories.

The paper therefore asks a central question: how can one design a flexible and universal memory system that supports long-term interactions without relying on static, predetermined memory operations? The proposed answer is to make memory itself *agentic*: instead of merely storing records, the system should dynamically construct structured notes, discover links among memories, and revise historical memories as new experiences arrive. This idea is inspired by the Zettelkasten method, where knowledge is maintained as a network of atomic notes and flexible links rather than as a rigid taxonomy.

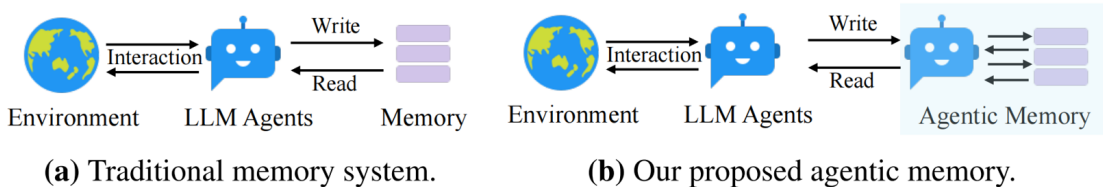


Figure 3.1: Comparison between a traditional memory system and A-Mem.

3.2 Overall architecture

A-Mem introduces a memory architecture centered on four stages: *note construction*, *link generation*, *memory evolution*, and *memory retrieval* [11]. The design philosophy is that every new interaction should be transformed into a structured memory note rather than kept as raw text only. Each note contains multiple semantic attributes that facilitate both retrieval and relationship discovery. When a new note is added, the system first finds semantically close neighbors, then reasons about meaningful links, and finally may revise the contextual representations of historical notes in light of the new arrival. Thus, memory writing is an active process rather than a passive append-only operation.

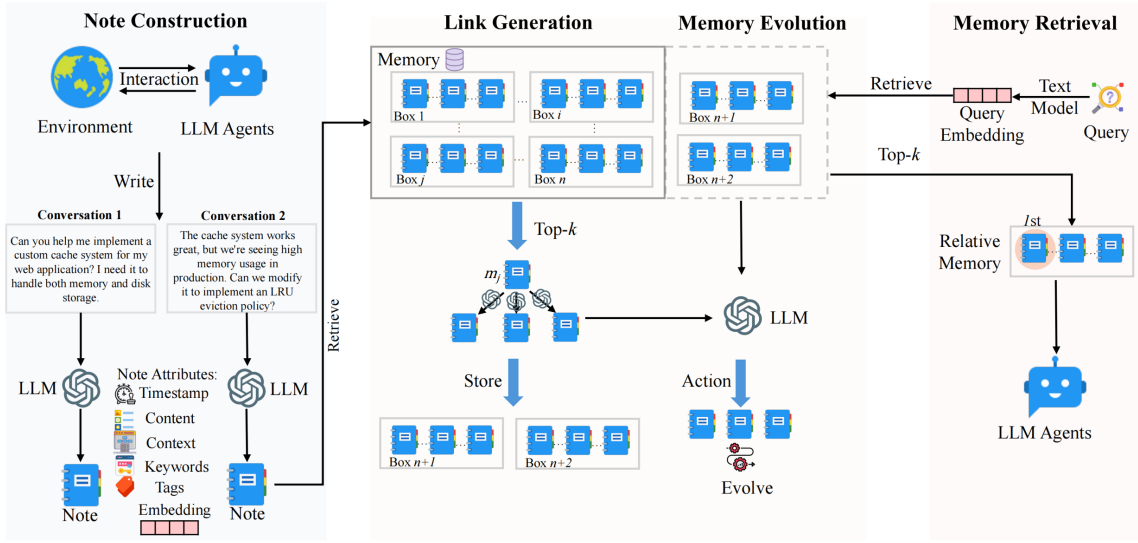


Figure 3.2: A-Mem architecture overview including note construction, link generation, memory evolution, and retrieval.

3.3 Method details

3.3.1 Structured note construction

For each new interaction, A-Mem constructs a comprehensive note with multiple attributes. Let the i -th memory be represented as:

$$m_i = \{c_i, t_i, K_i, G_i, X_i, e_i, L_i\}, \quad (3.1)$$

where c_i is the interaction content, t_i is the timestamp, K_i denotes keywords, G_i denotes tags, X_i is a contextual description, e_i is the embedding vector, and L_i is the set of links to other memories [11]. This formulation is important because it separates the raw memory content from derived metadata and relational structure.

The semantic attributes are generated by prompting an LLM on the current interaction content and timestamp:

$$K_i, G_i, X_i \leftarrow \text{LLM}(c_i \parallel t_i \parallel P_{s1}), \quad (3.2)$$

where P_{s1} denotes the prompt template for structured note construction. The embedding is then computed as

$$e_i = f_{\text{enc}}[\text{concat}(c_i, K_i, G_i, X_i)]. \quad (3.3)$$

This design choice means that the embedding reflects not only the raw interaction but also higher-level semantic summaries and tags, potentially improving retrieval quality and neighborhood discovery.

3.3.2 Link generation through retrieve-then-reason

Once a new note m_n is formed, A-Mem finds related historical memories. For each memory m_j in the memory base $\mathcal{M}$, a cosine similarity score is computed:

$$s_{n,j} = \frac{e_n \cdot e_j}{\|e_n\| \|e_j\|}. \quad (3.4)$$

The top- k nearest memories are selected as

$$\mathcal{M}_n^{\text{near}} = \{m_j \mid \text{rank}(s_{n,j}) \leq k, m_j \in \mathcal{M}\}. \quad (3.5)$$

Rather than directly using similarity edges as the final memory graph, A-Mem employs an LLM to reason about the relationship structure:

$$L_n \leftarrow \text{LLM}(m_n \parallel \mathcal{M}_n^{\text{near}} \parallel P_{s2}), \quad (3.6)$$

where P_{s2} is the prompt for link generation. This step is crucial in the paper’s design. Embedding retrieval provides scalable candidate selection, while the LLM adds semantic judgment about whether a meaningful connection exists and how it should be described.

3.3.3 Memory evolution

A particularly distinctive feature of A-Mem is that new memories can trigger updates to historical ones. For each nearby memory $m_j \in \mathcal{M}_n^{\text{near}}$, the system may revise its contextual description, keywords, tags, or links by conditioning on the new note and the surrounding neighborhood:

$$m_j^* \leftarrow \text{LLM}(m_n \parallel (\mathcal{M}_n^{\text{near}} \setminus m_j) \parallel m_j \parallel P_{s3}). \quad (3.7)$$

The updated note m_j^* then replaces m_j in the memory base. This operation captures the paper’s core claim that agent memory should not be static; rather, it should continuously refine its understanding as new experiences arrive. From an OS metaphor, this resembles a self-maintaining indexed store in which metadata and references are reorganized incrementally rather than only appended.

3.3.4 Memory retrieval

At inference time, a user query q is embedded as

$$e_q = f_{\text{enc}}(q), \quad (3.8)$$

and similarities to all stored notes are computed:

$$s_{q,i} = \frac{e_q \cdot e_i}{\|e_q\| \|e_i\|}. \quad (3.9)$$

The final retrieved set is

$$\mathcal{M}_{\text{retrieved}} = \{m_i \mid \text{rank}(s_{q,i}) \leq k, m_i \in \mathcal{M}\}. \quad (3.10)$$

These retrieved memories are then supplied to the LLM for answering or planning. The overall mechanism therefore combines semantic indexing, structured metadata, and evolving relational organization.

3.3.5 Algorithmic summary

Listing 3.1 summarizes the A-Mem lifecycle in pseudocode form.

Listing 3.1: A-Mem lifecycle pseudocode distilled from the paper.

```
# Write-side
input: interaction content c_n, timestamp t_n
(K_n, G_n, X_n) <- LLM(c_n || t_n || P_s1)
e_n <- f_enc(concat(c_n, K_n, G_n, X_n))
m_n <- {c_n, t_n, K_n, G_n, X_n, e_n, L_n = {}}

# Link generation
for each m_j in M:
  s_{n,j} <- cosine(e_n, e_j)
M_near <- TopK(M, key=s_{n,j}, k)
L_n <- LLM(m_n || M_near || P_s2)

# Memory evolution
for each m_j in M_near:
  m_j^* <- LLM(m_n || (M_near \ m_j) || m_j || P_s3)
```

```

    replace m_j with m_j^* in M

insert m_n into M

# Read-side
input: query q
e_q <- f_enc(q)
return TopK(M, key=cosine(e_q, e_i), k)

```

3.4 Experimental settings

The paper evaluates A-Mem primarily on the LoCoMo benchmark, which is designed to test long-term conversational memory in LLM agents [8]. The evaluation reports F1 and BLEU-1 across multiple QA categories, including multi-hop, temporal, open-domain, single-hop, and adversarial questions. The paper also tracks average token length per question, highlighting the practical efficiency implications of different memory systems.

The main baselines include LoCoMo’s method baseline, ReadAgent [5], Memory-Bank [14], and MemGPT [10]. A-Mem is evaluated across several foundation model families, including GPT, Qwen2.5, and Llama3.2 variants, demonstrating that the memory mechanism is intended to be model-agnostic.

3.5 Experimental results and discussion

Table 3.1 reproduces the detailed experimental results. A-Mem achieves strong performance across categories and frequently improves the average ranking while using dramatically fewer tokens than methods that rely on replaying large portions of dialogue history. Particularly notable is its improvement on temporal and multi-hop categories in several model settings, suggesting that structured notes and dynamic links help the model recover dependencies that span long interaction histories.

The paper also studies the effect of the retrieval hyperparameter k . The reported trend is intuitive yet important: increasing k can provide richer context, but beyond a certain point it may also inject noise and enlarge prompt length, creating a trade-off between information coverage and retrieval precision. The corresponding figure is shown in Fig. 3.3.

In addition, the paper includes a scaling analysis. As shown in Table 3.2, space usage scales linearly with memory size, while retrieval remains efficient even when the number of memories grows to one million. This observation is relevant from a systems angle: A-Mem achieves sophisticated semantic organization without making read-time retrieval prohibitively expensive.

Table 3.1: Representative A-Mem results on LoCoMo.

Family	Model	Method	Avg. Rank (F1) ↓	Avg. Rank (BLEU-1) ↓	Token Length
GPT	4o-mini	LOCOMO	2.4	2.4	16,910
GPT	4o-mini	MemGPT	2.4	2.4	16,977
GPT	4o-mini	A-Mem	1.2	1.2	2,520
GPT	4o	LOCOMO	2.0	2.0	16,910
GPT	4o	A-Mem	1.6	1.6	1,216
Qwen2.5	1.5b	MemoryBank	2.6	2.6	284
Qwen2.5	1.5b	A-Mem	1.0	1.0	1,300
Qwen2.5	3b	LOCOMO	3.2	3.2	16,910
Qwen2.5	3b	A-Mem	1.0	1.0	1,137
Llama3.2	1b	MemoryBank	2.0	2.0	274
Llama3.2	1b	A-Mem	1.0	1.0	1,376
Llama3.2	3b	LOCOMO	2.8	2.8	16,910
Llama3.2	3b	A-Mem	1.0	1.0	1,126

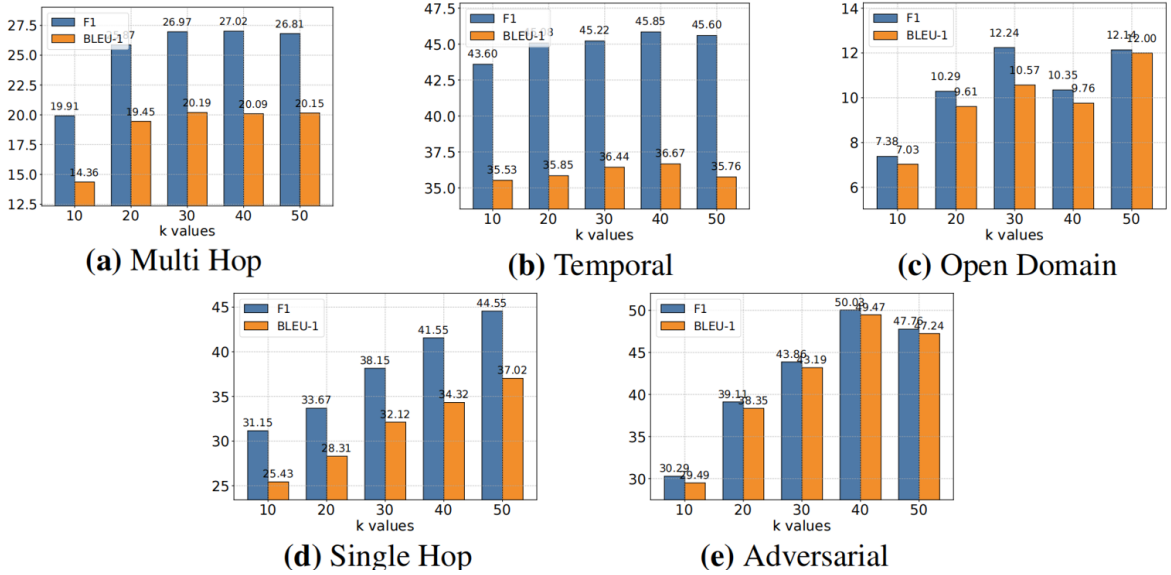


Figure 3.3: Impact of retrieval parameter k across task categories.

Table 3.2: Scaling analysis for different memory systems.

Memory Size	Method	Memory Usage (MB)	Retrieval Time (μs)
1,000	A-Mem	1.46	0.31 ± 0.30
	MemoryBank	1.46	0.24 ± 0.20
	ReadAgent	1.46	43.62 ± 8.47
10,000	A-Mem	14.65	0.38 ± 0.25
	MemoryBank	14.65	0.26 ± 0.13
	ReadAgent	14.65	484.45 ± 93.86
100,000	A-Mem	146.48	1.40 ± 0.49
	MemoryBank	146.48	0.78 ± 0.26
	ReadAgent	146.48	$6,682.22 \pm 111.63$
1,000,000	A-Mem	1464.84	3.70 ± 0.74
	MemoryBank	1464.84	1.91 ± 0.31
	ReadAgent	1464.84	$120,069.68 \pm 1,673.39$

Overall, A-Mem offers a compelling answer to the survey’s dynamic perspective on memory. It treats memory not just as a store to retrieve from, but as an evolving semantic network. Its main strength lies in flexible organization and continual refinement. Its main systems cost, however, is that write-time processing becomes relatively sophisticated, since note creation, link generation, and memory evolution all require LLM-assisted operations.

Chapter 4

MemoryOS: A Memory Operating System for AI Agents

4.1 Motivation

MemoryOS is motivated by the observation that fixed context windows lead to severe deficiencies in long-term memory and personalization for AI agents [4]. The authors argue that existing memory mechanisms often optimize only isolated components, such as retrieval or storage, but fail to provide an integrated memory management framework. Inspired directly by memory management in operating systems, they propose a *Memory Operating System* that coordinates storage, update, retrieval, and response generation in a unified architecture.

The motivation is especially OS-like in two senses. First, the system distinguishes multiple memory tiers with different roles and persistence properties. Second, it defines explicit policies for moving information across these tiers, including FIFO-based transfer, segmentation by topic, and heat-based promotion or deletion. This is a much more explicit systems abstraction than simply storing dialogue history and retrieving similar passages.

4.2 Architecture overview

MemoryOS consists of four main modules: *Memory Storage*, *Memory Updating*, *Memory Retrieval*, and *Memory Generation* [4]. The storage architecture itself is hierarchical, containing *Short-Term Memory* (STM), *Mid-Term Memory* (MTM), and *Long-Term Personal Memory* (LPM).

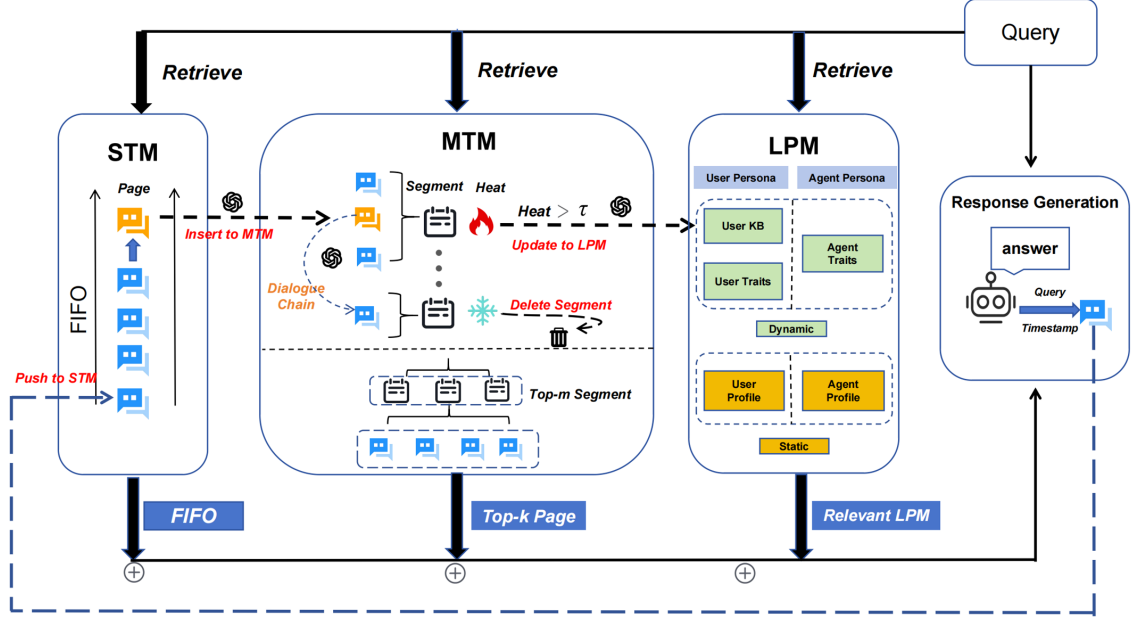


Figure 4.1: MemoryOS architecture with STM, MTM, and LPM, plus storage, update, retrieval, and generation modules.

4.3 Method details

4.3.1 Short-Term Memory and dialogue-chain pages

STM stores recent interactions as pages that contain a query Q_i , response R_i , and timestamp T_i [4]. To improve coherence, the system constructs a dialogue chain representation for each page:

$$\text{page}_i^{\text{chain}} = \{Q_i, R_i, T_i, \text{meta}_i^{\text{chain}}\}, \quad (4.1)$$

where $\text{meta}_i^{\text{chain}}$ is generated by an LLM and captures whether the page should continue the current dialogue chain or start a new one, together with a summary of the chain. This design preserves local conversational continuity while preventing short-term memory from degenerating into an unstructured queue of turns.

4.3.2 Mid-Term Memory with segmented paging

MTM acts as a larger but slower store that groups related pages into topic-based segments. The authors explicitly describe this as a segmented paging design inspired by operating systems [4]. A segment is defined by topic coherence, and a page is assigned to a segment if its matching score exceeds a threshold θ :

$$\text{segment}_i = \{\text{page}_j \mid F_{\text{score}}(\text{page}_j, \text{segment}_i) > \theta\}. \quad (4.2)$$

The score function combines semantic similarity and keyword overlap:

$$F_{\text{score}} = \cos(e_s, e_p) + F_{\text{Jaccard}}(K_s, K_p), \quad (4.3)$$

where the Jaccard term is

$$F_{\text{Jaccard}} = \frac{|K_s \cap K_p|}{|K_s \cup K_p|}. \quad (4.4)$$

This combination of dense semantic similarity and discrete keyword overlap is a sensible compromise: semantic embeddings capture broad topical similarity, while keyword overlap helps preserve sharper topic boundaries.

4.3.3 Long-Term Personal Memory

LPM stores persistent user- and agent-specific information. MemoryOS divides this long-term memory into user profile, agent profile, user knowledge base, and agent personality traits [4]. In the paper, the trait component is represented as a 90-dimensional vector that covers needs, personality, alignment, and interest dimensions. This makes MemoryOS not only a memory system for task continuity, but also a personalization system.

4.3.4 Memory updating and heat-based management

One of the most OS-inspired parts of MemoryOS is its update mechanism. STM is implemented as a fixed-length FIFO queue. When STM overflows, the oldest page is moved into MTM. This STM→MTM transfer mirrors the behavior of moving less immediately needed data from a fast tier to a larger slower tier.

Within MTM, segments are further managed by a *heat* score:

$$\text{Heat} = \alpha \cdot N_{\text{visit}} + \beta \cdot L_{\text{interaction}} + \gamma \cdot R_{\text{recency}}, \quad (4.5)$$

where N_{visit} is the number of retrieval visits, $L_{\text{interaction}}$ is the number of pages in the segment, and the recency term is defined as

$$R_{\text{recency}} = \exp\left(-\frac{\Delta t}{\mu}\right). \quad (4.6)$$

Here Δt denotes time since last interaction, and μ is a decay constant. Segments with low heat may be deleted when MTM capacity is exceeded, whereas high-heat segments whose score exceeds a threshold τ can be promoted into long-term personal memory. This policy resembles a combination of recency-aware caching and importance-based promotion.

4.3.5 Retrieval and response generation

During inference, MemoryOS retrieves relevant memory from all three tiers:

$$F_{\text{Retrieval}}(\text{STM}, \text{MTM}, \text{LPM} \mid Q), \quad (4.7)$$

where Q is the user query. STM retrieval returns all pages in the short-term queue, MTM retrieval performs a two-stage process of segment selection followed by page selection, and LPM retrieval returns relevant profile or knowledge-base entries. The retrieved content is then integrated into the prompt used for generation. This design explicitly recognizes that different tiers serve different roles: STM for immediate continuity, MTM for topical episodic recall, and LPM for persistent personalization.

4.3.6 Algorithmic summary

Listing 4.1 summarizes the lifecycle of MemoryOS.

Listing 4.1: MemoryOS lifecycle pseudocode distilled from the paper.

```
# STM: fixed-length FIFO queue
# MTM: segmented paging + heat-based eviction/promotion
# LPM: persistent persona memory

on_new_dialogue_page(page = {Q, R, T}):
    append page to STM_queue
    update dialogue chain metadata via LLM
    if len(STM_queue) > STM_capacity:
        oldest_page <- pop_front(STM_queue)
        insert_into_MTM(oldest_page)

insert_into_MTM(page):
    seg <- argmax_s F_score(page, segment_s)
    if F_score(page, seg) > theta:
        merge page into seg
    else:
        create new segment
    update heat metrics
    evict low-heat segments if MTM exceeds capacity
    if Heat(seg) > tau:
        promote seg to LPM

retrieve(Q):
    ctx_STM <- all pages in STM_queue
    segs <- top_m segments by F_score(Q, segment)
    pages <- top_k pages within segs by semantic similarity
    lpm_entries <- top relevant entries from LPM
```

```
return {ctx_STM, pages, lpm_entries}
```

4.4 Experimental settings

MemoryOS is evaluated on both GVD and LoCoMo [4]. On LoCoMo, it reports F1 and BLEU-1 across categories, while on GVD it uses an LLM-based scoring protocol. The paper compares against TiM, MemoryBank, MemGPT, and A-Mem. Notably, the paper also includes an *A-Mem** re-implementation under the same environment to provide a fairer comparison.

The reported hyperparameters reveal a clearly systematized design: the STM queue length is set to 7, the maximum MTM segment length to 2000, the persona capacities to 100, the promotion threshold τ to 5, the score threshold θ to 0.6, and the retrieval top- m and top- k values to 5 segments and 5 or 10 pages depending on the benchmark [4]. These details reinforce the paper’s operating-systems flavor, because the behavior of the memory subsystem is explicitly governed by configurable management policies.

4.5 Experimental results and discussion

Table 4.1 reproduces the detailed LoCoMo results. MemoryOS achieves the best average ranking in both reported model settings. Particularly strong gains appear in the temporal and open-domain categories, suggesting that the hierarchical design and the explicit promotion into long-term persona memory are especially useful for maintaining continuity over extended dialogues.

Table 4.1: LoCoMo comparison for MemoryOS.

Model	Method	Single Hop		Multi Hop		Temporal		Open Domain		Avg. Rank $\downarrow$ (F1)	Avg. Rank $\downarrow$ (BLEU-1)
		F1 $\uparrow$	BLEU-1 $\uparrow$	F1 $\uparrow$	BLEU-1 $\uparrow$	F1 $\uparrow$	BLEU-1 $\uparrow$	F1 $\uparrow$	BLEU-1 $\uparrow$		
GPT-4o-mini	TiM	16.25	13.12	18.43	17.35	8.35	7.32	23.74	22.05	3.8	4.0
	MemoryBank	5.00	4.77	9.68	6.99	5.56	5.94	6.61	5.16	5.0	5.0
	MemGPT	26.65	17.72	25.52	19.44	9.15	7.44	41.04	34.34	2.2	2.5
	A-Mem	27.02	20.09	45.85	36.67	12.14	12.00	44.65	37.06	–	–
	A-Mem*	22.61	15.25	33.23	29.11	8.04	7.81	34.13	27.73	3.0	2.5
	Ours	35.27	25.22	41.15	30.76	20.02	16.52	48.62	42.99	1.0	1.0
	Improvement (%)	32.35%	42.33%	23.83%	5.67%	118.80%	111.52%	18.47%	25.19%	–	–
Qwen2.5-3B	TiM	4.37	5.01	2.54	3.21	6.20	5.37	6.35	7.34	4.3	3.5
	MemoryBank	3.60	3.39	1.72	1.97	6.63	6.58	4.11	3.32	4.8	4.8
	MemGPT	5.07	4.31	2.94	2.95	7.04	7.10	7.26	5.52	2.8	3.8
	A-Mem	12.57	9.01	27.59	25.07	7.12	7.28	17.23	13.12	–	–
	A-Mem*	10.31	8.76	16.31	11.07	6.94	7.31	12.34	10.62	2.3	2.0
	Ours	23.26	15.39	21.44	14.95	10.18	8.18	26.23	22.39	1.0	1.0
	Improvement (%)	125.61%	75.68%	31.45%	35.05%	46.69%	11.90%	112.56%	110.83%	–	–

The paper also reports ablations demonstrating that removing elements of the hierarchical design degrades performance, which is shown in Fig. 4.2.

From a professor-level assessment, the main contribution of MemoryOS is conceptual coherence. Unlike approaches that improve only retrieval or only summarization, MemoryOS presents a full-stack memory abstraction with clear tiers and explicit movement

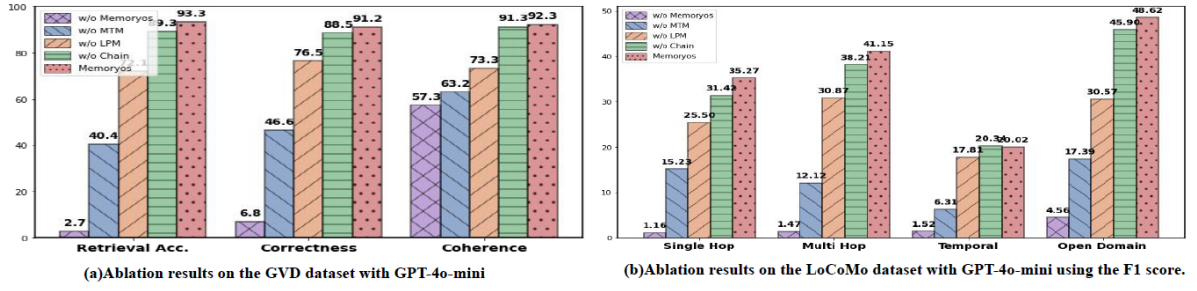


Figure 4.2: Ablation study on GVD and LoCoMo benchmark datasets.

policies. Its strongest point is not merely the numerical gains, but the fact that it offers a principled systems framework for how memory can be organized and managed in long-horizon agents.

Chapter 5

Agentic Plan Caching: Test-Time Memory for Serving Efficiency

5.1 Motivation

APC addresses a different but equally important aspect of memory: efficiency during serving. The paper observes that many LLM-based agents follow a *Plan-Act* loop, often rooted in the ReAct paradigm [12]. In such agents, the planning stage is frequently the most expensive part because it involves multiple calls to a large language model, often with chain-of-thought-style reasoning and repeated refinement. Existing caching approaches, such as context caching or semantic caching, are mainly designed for chatbot serving and are less effective when outputs depend strongly on external data or environment state [13].

The paper’s central insight is that while the details of a task instance may vary, the *high-level plan structure* often recurs across semantically similar requests. Therefore, instead of caching final responses, APC caches reusable *plan templates*. This transforms memory into a test-time optimization mechanism: historical experience is exploited not primarily to improve answer quality, but to reduce planning cost and latency while maintaining near-equivalent performance.

5.2 Method overview

APC maintains a cache of pairs of the form (keyword, plan template). Given a new request, it first extracts a high-level keyword representing the task intent. If the keyword matches an existing cache entry, APC treats this as a cache hit and adapts the stored plan template to the current context using a smaller planner model. If the keyword is absent, APC invokes a larger planner model to generate a new plan, executes it, and then extracts a generalized reusable template to store for future use [13]. This architecture is

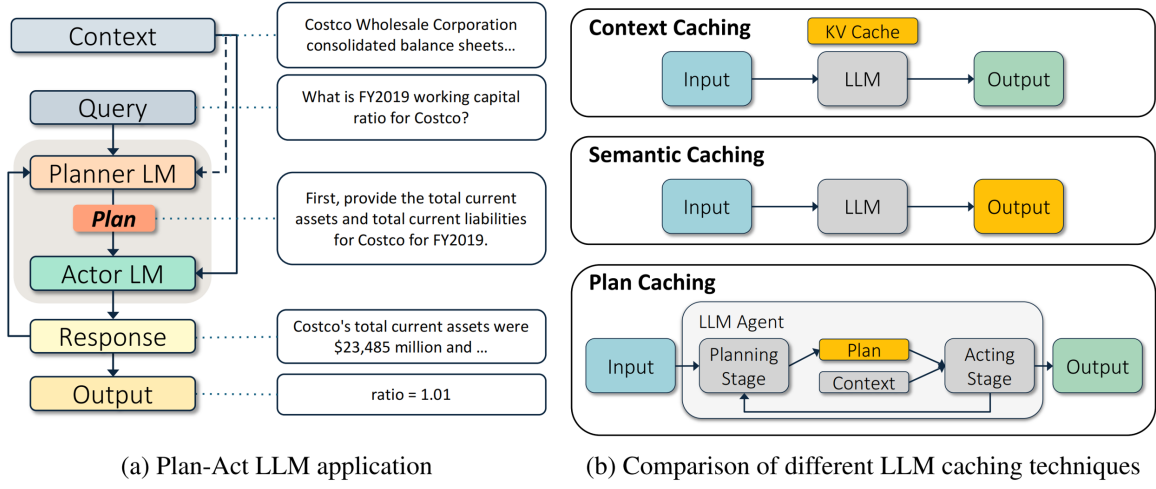


Figure 5.1: Motivation for APC: limitations of standard caching for Plan-Act agents.

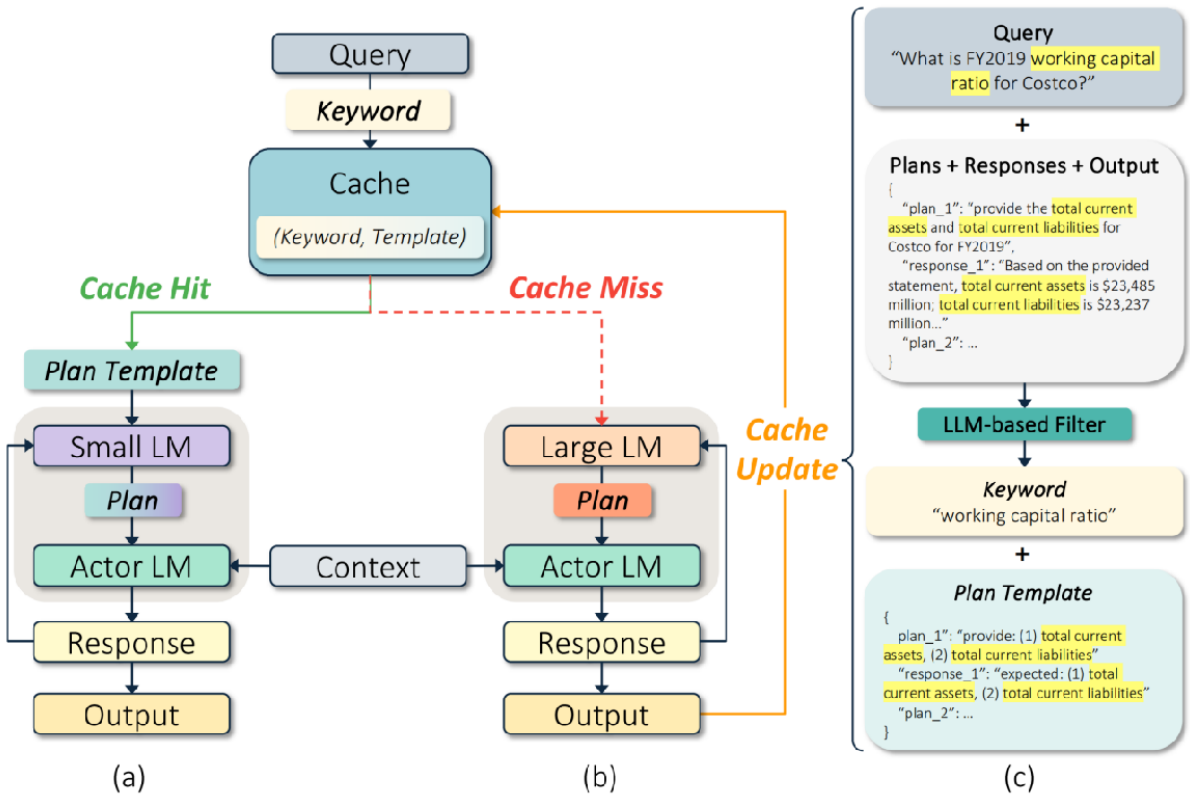


Figure 5.2: Agentic plan caching framework.

illustrated in Fig. 5.2.

5.3 Method details

5.3.1 Keyword-based cache lookup

A notable design decision in APC is to avoid relying purely on raw query embeddings for cache lookup. The paper argues that queries often contain instance-specific details, so embedding similarity may incorrectly group tasks that differ in underlying intent or fail to match tasks that share the same high-level workflow [13]. Instead, APC uses a lightweight LLM to extract a concise keyword describing the core intent. Exact or fuzzy matching on this keyword then determines whether a plan template can be reused. As shown in Fig. 5.3, using keyword matching has better results.

This design separates *intent* from *context*. In systems terms, it is similar to normalizing requests before cache lookup so that the cache key captures reusable structure rather than transient input details.

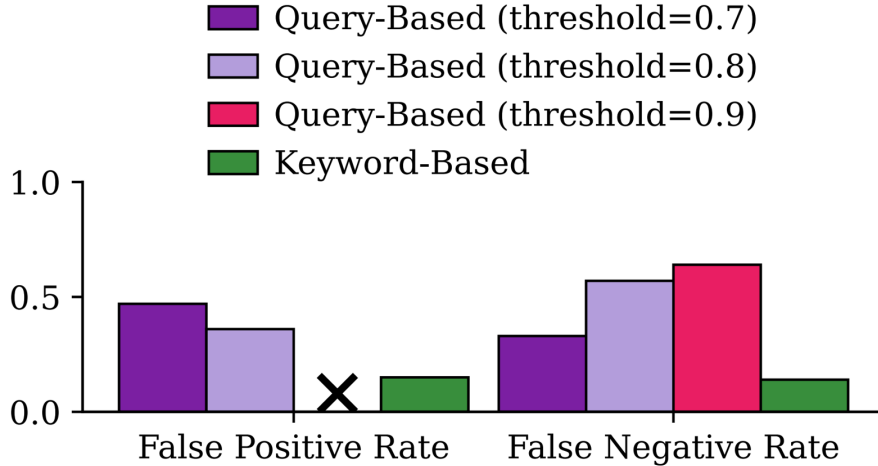


Figure 5.3: Query-based versus keyword-based cache search.

5.3.2 Plan template extraction

When a cache miss occurs, APC executes the full planning process using a large planner model and an actor model. After completion, the system extracts a reusable template from the execution trace. According to the paper, this involves a two-step filtering process: a rule-based filter retains the critical planning structure, and a lightweight LLM-based filter removes context-specific details such as named entities or numeric values that should not be hard-coded into the template [13]. The result is a generalized plan template that captures workflow structure while remaining adaptable to future instances.

5.3.3 Template adaptation on cache hits

On a cache hit, APC uses a smaller planner model to adapt the cached template to the current context. This is a key engineering contribution: the system avoids repeating the full cost of large-model planning while still tailoring the plan to current data or environmental conditions. Therefore, the cache stores not final answers but reusable reasoning skeletons.

5.3.4 Algorithmic summary

Listing 5.1 summarizes APC in pseudocode form.

Listing 5.1: APC pseudocode distilled from the paper.

```
# cache stores (keyword -> plan_template)
on_request(query, context):
  kw <- SmallLM.keyword_extract(query)
  if kw in cache:
    template <- cache[kw]
    plan <- SmallPlannerLM.adapt(template, context)
  else:
    plan <- LargePlannerLM.plan(query, context)

  answer <- ActorLM.execute(plan, context)
  while not LargePlannerLM.is_done(answer):
    plan <- LargePlannerLM.plan_refine(query, context, answer)
    answer <- ActorLM.execute(plan, context)

  if kw not in cache:
    exec_log <- {plans, tool calls, intermediate outputs, answer}
    template <- filter_to_plan_template(exec_log)
    cache[kw] <- template
  return answer
```

5.4 Experimental settings

APC is evaluated on five workloads: FinanceBench [3], QASPER [1], TabMWP [7], AIME [15], and GAIA [9]. These tasks cover both long-context question answering and tool-using multi-step problem solving. The evaluation considers not only application performance, but also latency and monetary cost using API pricing and token accounting [13]. This is significant because APC is primarily an efficiency-oriented memory mechanism rather than a capability-only memory mechanism.

The headline results reported by the paper are that APC reduces cost by 50.31% and latency by 27.28% on average, while retaining 96.61% of the accuracy of the accuracy-

optimal baseline [13]. This directly supports the paper’s claim that plan-level reuse is a viable serving-time optimization.

5.5 Experimental results and discussion

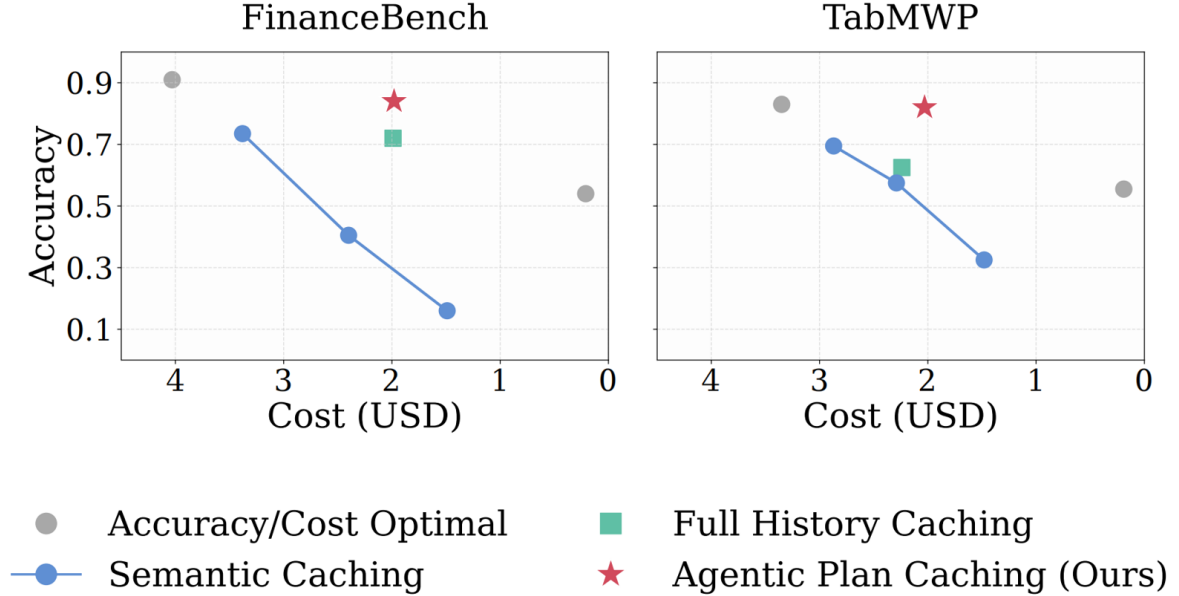


Figure 5.4: Main APC results across baselines.

Table 5.1 reproduces the cost breakdown. The table shows that the cache overhead is very small relative to the dominant planner cost, typically around 1% of total cost. This is an important systems-level result, because it indicates that the additional infrastructure required for keyword extraction and cache generation does not erase the gains from plan reuse.

Table 5.1: Cost breakdown for APC on FinanceBench and TabMWP .

Component	FinanceBench		TabMWP	
	Main Results	Worst Case	Main Results	Worst Case
Large Planner LM	\$1.7544 (94.17%)	\$3.9227 (97.36%)	\$1.9823 (97.76%)	\$3.3292 (98.33%)
Small Planner LM	\$0.0168 (0.90%)	–	\$0.0095 (0.47%)	–
Actor LM	\$0.0705 (3.78%)	\$0.0529 (1.31%)	\$0.0170 (0.84%)	\$0.0128 (0.38%)
Cache Overhead	\$0.0213 (1.15%)	\$0.0535 (1.33%)	\$0.0190 (0.93%)	\$0.0438 (1.29%)
- Keyword Extraction	\$0.0050 (0.27%)	\$0.0050 (0.13%)	\$0.0025 (0.12%)	\$0.0025 (0.07%)
- Cache Generation	\$0.0163 (0.88%)	\$0.0485 (1.20%)	\$0.0165 (0.81%)	\$0.0413 (1.22%)
Total	\$1.8630 (100%)	\$4.0291 (100%)	\$2.0278 (100%)	\$3.3858 (100%)

The paper also studies fuzzy keyword matching, summarized in Table 5.2. As the similarity threshold decreases, the hit rate increases and cost decreases, but accuracy

begins to drift. This makes clear that APC also involves a systems trade-off similar to cache design in classical systems: more aggressive reuse can reduce overhead but may introduce mismatches. The accuracy comparison across caching methods is reported in Fig. 5.5.

Table 5.2: Fuzzy keyword matching results in APC under different similarity thresholds.

Similarity Threshold	Hit Rate	Cost	Accuracy	Planning Latency (s)	Total Latency (s)
= 100%	46%	\$1.86	85.50%	1011.82	1424.82
> 80%	54%	\$1.15	83.00%	875.31	1219.73
> 60%	64%	\$0.93	77.00%	720.29	1044.50

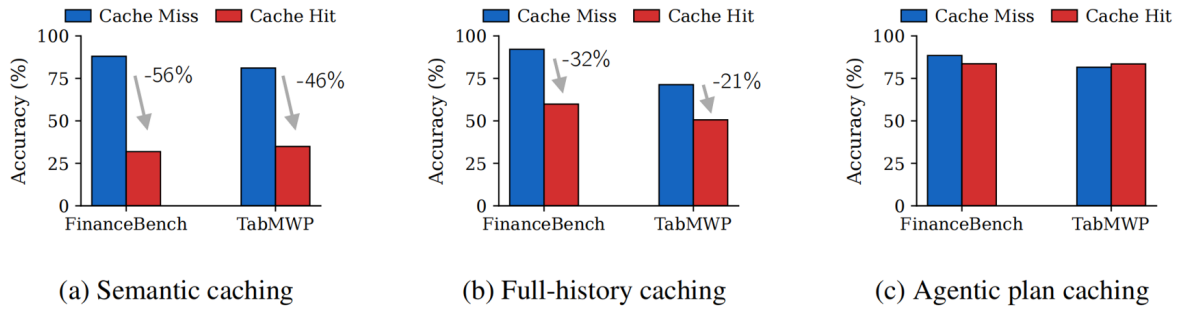


Figure 5.5: Accuracy comparison across caching methods.

From an OS perspective, APC plays a role analogous to caching reusable execution structures rather than reusable outputs. This is conceptually elegant because it recognizes that what is costly in agent systems is often not only data access but also repeated reasoning. APC therefore broadens the notion of memory from storing *information* to storing *procedures*. It is highly complementary to A-Mem and MemoryOS, which focus more on capability and continuity.

Chapter 6

Conclusion and Future Work

This report examined memory management for LLM-based agents through the combined lens of a recent survey and three representative papers: A-Mem, MemoryOS, and APC. Taken together, these works show that memory in agent systems is no longer adequately described as simple history retention or document retrieval. Instead, it is increasingly being designed as a managed subsystem with explicit structures, policies, and trade-offs.

A-Mem contributes a view of memory as a self-organizing semantic note network. Its main innovation lies in transforming interactions into structured memory notes, reasoning about semantic links among them, and allowing new experiences to revise historical memories. In terms of the survey’s taxonomy, A-Mem strongly emphasizes dynamic experiential memory and shows how memory can *evolve*, not merely accumulate.

MemoryOS contributes the clearest operating-systems abstraction. By introducing short-term, mid-term, and long-term personal memory, along with dialogue-chain pages, segmented paging, and heat-based promotion/eviction policies, it treats memory as a hierarchical resource that must be managed across tiers. It thereby provides an explicit bridge between classical OS ideas and modern agent memory design. Its strongest contribution is architectural coherence: it shows how storage, update, retrieval, and generation can be integrated into a single memory framework.

APC contributes an efficiency-oriented notion of memory. Rather than focusing only on capability or personalization, it treats historical planning traces as reusable templates that can reduce the repeated cost of reasoning at test time. This reframes memory as a mechanism for computational reuse, analogous to caching in classical systems. By separating task intent from context and caching plan templates rather than final responses, APC demonstrates that agent memory can directly optimize serving efficiency without severe loss of performance.

From a higher-level academic perspective, the three systems are not redundant alternatives; rather, they occupy complementary points in the design space. A-Mem is strongest in semantic organization and continual reinterpretation. MemoryOS is strongest in hierarchical management and long-term personalization. APC is strongest in resource-

efficient reuse. Together, they suggest that future agent memory systems should integrate all three principles: *structured representation*, *explicit management policies*, and *efficiency-aware reuse*. In other words, memory should be treated as a first-class subsystem for agentic intelligence, much as memory hierarchies, segmentation, paging, and caching are foundational subsystems in operating systems.

Despite these advances, current agent memory systems still have several limitations. First, many methods remain heavily dependent on hand-designed memory structures and heuristic policies. For example, thresholds for memory promotion, deletion, link generation, or cache reuse are often manually specified, which may limit their generalization across tasks, users, and deployment environments. Second, existing evaluations are still relatively narrow. Many benchmarks focus on long-term conversational QA or isolated planning tasks, while real-world agents must handle noisy, multimodal, and dynamically changing environments. Third, the reliability and safety of memory updates remain insufficiently understood. Incorrectly written, outdated, or over-retrieved memories may accumulate over time and eventually mislead the agent. Finally, the computational overhead of sophisticated memory writing and updating is still a practical concern, especially for systems that repeatedly invoke LLMs to summarize, link, or revise memories.

These limitations point to several promising directions for future research. One direction is the unification of capability-oriented and efficiency-oriented memory, such that semantic memory systems like A-Mem or MemoryOS could also cache reusable reasoning fragments, plans, or skills. A second direction is stronger multimodal memory, where visual observations, textual interactions, tool-use traces, and structured environment states are managed under a common framework. A third direction is more principled and adaptive policy design, for example learning promotion, deletion, compression, and cache-hit decisions automatically rather than relying solely on hand-designed rules. A fourth direction is the development of more comprehensive evaluation protocols that measure not only retrieval accuracy, but also long-term consistency, robustness to outdated memory, computational cost, and safety under continuous memory evolution. These directions indicate that agent memory is likely to become a major interdisciplinary theme connecting NLP, machine learning systems, and operating systems.

Bibliography

- [1] Dasigi, P., Lo, K., Beltagy, I., Cohan, A., Smith, N.A., Gardner, M.: A Dataset of Information-Seeking Questions and Answers Anchored in Research Papers. arXiv preprint arXiv:2105.03011 (2021)
- [2] Hu, Y., Liu, S., Yue, Y., Zhang, G., Liu, B., Zhu, F., Lin, J., Guo, H., Dou, S., Xi, Z., Jin, S., Tan, J., Yin, Y., Liu, J., Zhang, Z., Sun, Z., Zhu, Y., Sun, H., Peng, B., Cheng, Z., Fan, X., Guo, J., Yu, X., Zhou, Z., Hu, Z., Huo, J., Wang, J., Niu, Y., Wang, Y., Yin, Z., Hu, X., Liao, Y., Li, Q., Wang, K., Zhou, W., Liu, Y., Cheng, D., Zhang, Q., Gui, T., Pan, S., Zhang, Y., Torr, P., Dou, Z., Wen, J.-R., Huang, X., Jiang, Y.-G., Yan, S.: Memory in the Age of AI Agents: A Survey – Forms, Functions and Dynamics. arXiv preprint arXiv:2512.13564v2 (2026)
- [3] Islam, P., Kannappan, A., Kiela, D., Qian, R., Scherrer, N., Vidgen, B.: FinanceBench: A New Benchmark for Financial Question Answering. arXiv preprint arXiv:2311.11944 (2023)
- [4] Kang, J., Ji, M., Zhao, Z., Bai, T.: Memory OS of AI Agent. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 25961–25970 (2025)
- [5] Lee, K.-H., Chen, X., Furuta, H., Canny, J., Fischer, I.: A Human-Inspired Reading Agent with Gist Memory of Very Long Contexts. arXiv preprint arXiv:2402.09727 (2024)
- [6] Liu, L., Yang, X., Shen, Y., Hu, B., Zhang, Z., Gu, J., Zhang, G.: Think-in-Memory: Recalling and Post-Thinking Enable LLMs with Long-Term Memory. arXiv preprint arXiv:2311.08719 (2023)
- [7] Lu, P., Qiu, L., Chang, K.-W., Wu, Y.N., Zhu, S.-C., Rajpurohit, T., Clark, P., Kalyan, A.: Dynamic Prompt Learning via Policy Gradient for Semi-Structured Mathematical Reasoning. arXiv preprint arXiv:2209.14610 (2022)
- [8] Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., Fang, Y.: Evaluating Very Long-Term Conversational Memory of LLM Agents. In: Proceedings of

- the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 13851–13870 (2024)
- [9] Mialon, G., Fourrier, C., Wolf, T., LeCun, Y., Scialom, T.: GAIA: A Benchmark for General AI Assistants. In: International Conference on Learning Representations (2023)
 - [10] Packer, C., Fang, V., Patil, S.G., Lin, K., Wooders, S., Gonzalez, J.E.: MemGPT: Towards LLMs as Operating Systems. arXiv preprint arXiv:2310.08560 (2023)
 - [11] Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., Zhang, Y.: A-Mem: Agentic Memory for LLM Agents. In: Advances in Neural Information Processing Systems (NeurIPS) (2025)
 - [12] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing Reasoning and Acting in Language Models. In: International Conference on Learning Representations (2023)
 - [13] Zhang, Q., Wornow, M., Wan, G., Olukotun, K.: Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents. In: Advances in Neural Information Processing Systems (NeurIPS) (2025)
 - [14] Zhong, W., Guo, L., Gao, Q., Ye, H., Wang, Y.: MemoryBank: Enhancing Large Language Models with Long-Term Memory. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, pp. 19724–19731 (2024)
 - [15] Mathematical Association of America: American Invitational Mathematics Examination (AIME). (2026)